# EXPERIMENT 14

**Exploring Pro-Code Extension of a Zoho Creator Application using Custom Functions and Zoho Catalyst**

Project: Patient Appointment Scheduling System | Platform: Zoho Creator

Saranya V. | Register No: 2116241501188 | AIML | Rajalakshmi Engineering College

## Abstract

This experiment explores how the Patient Appointment Scheduling System can be extended beyond its low-code features using reusable Deluge functions and a serverless service. The current project includes an Appointment Booking Form, appointment reports, dashboard components and an existing Auto-Calculate Net Fee workflow. A proposed Zoho Catalyst function calculates the net payable consultation fee from the consultation fee and insurance coverage percentage. The function and its Creator call are provided as an implementation design; deployment depends on the availability of Catalyst and the required connection in the account.

## 1. Aim

To explore pro-code extension of the Patient Appointment Scheduling System using custom functions, Zoho Creator developer tools and a proposed Zoho Catalyst serverless function.

## Note on Execution

The supplied screenshots verify the appointment form, an existing Deluge fee-calculation workflow, the Settings page and the appointment report. They do not show a deployed Catalyst function or a completed external connection. Therefore, the Catalyst code and invokeUrl integration are documented as a design and must not be reported as deployed or tested unless completed in the account.

## 2. Concepts

### Custom functions

Reusable Deluge functions can centralize business logic and be called from workflows, buttons or schedules. For this project, fee-calculation logic can be kept reusable rather than repeated in several workflows.

### Developer tools

The Settings page exposes Schema Builder, Application IDE, Widgets and Variables. These tools help inspect the application structure and support more advanced customization.

### Zoho Catalyst

Zoho Catalyst is a serverless development platform. A function can be exposed over HTTPS and called from Creator through invokeUrl and a configured connection, subject to account access and platform requirements.

### Project use case

The proposed function receives a consultation fee and insurance coverage percentage, validates the inputs, calculates the net payable fee, and returns the result as JSON for Creator to display or store.

## 3. Procedure

1. Open the Patient Appointment Scheduling System in Zoho Creator and select Workflow.

2. Review the existing Auto-Calculate Net Fee workflow. Confirm the field link names used by the current Deluge script, including consultation_fee and insurance_coverage_percent.

3. Open Settings and review Developer Tools: Schema Builder, Application IDE, Widgets and Variables.

4. If the account supports Zoho Catalyst, create a Catalyst project and an Advanced I/O function using the Node.js example in Section 4. Deploy the function and note its HTTPS endpoint.

5. Create a secure Creator connection for the Catalyst endpoint according to the platform's current connection setup. Do not place secrets or access tokens directly in the code.

6. Add the Deluge invokeUrl call to the appropriate Appointment Booking Form workflow. Pass the consultation fee and insurance coverage percentage to the endpoint.

7. Read the returned netFee value and assign it to the relevant net payable fee field, after confirming that the field link name matches the form.

8. Test with a consultation fee of 1000 and coverage of 20 percent. The expected net payable fee is 800. Also test missing, negative and invalid values.

9. If Catalyst or connections are unavailable in the current plan, retain the existing Deluge workflow and document the Catalyst integration as a proposed extension rather than a completed deployment.

## 4. Deluge / Configuration Code

Proposed Catalyst Node.js function:

```
// Proposed Zoho Catalyst Advanced I/O function (Node.js)
const express = require("express");
const app = express();

app.get("/net-fee", (req, res) => {
  const consultationFee = Number(req.query.consultationFee);
  const coveragePercent = Number(req.query.coveragePercent);

  if (!Number.isFinite(consultationFee) ||
      !Number.isFinite(coveragePercent) ||
      consultationFee < 0 ||
      coveragePercent < 0 ||
      coveragePercent > 100) {
    return res.status(400).json({
      error: "Enter a valid fee and coverage from 0 to 100."
    });
  }
```

```
  const netFee = consultationFee *
                 (1 - coveragePercent / 100);

  return res.json({
    consultationFee,
    coveragePercent,
    netFee: Number(netFee.toFixed(2))
  });
});

module.exports = app;
```

Proposed Creator Deluge call:

```
// Proposed Deluge integration after confirming field link names
fee = input.consultation_fee;
coverage = input.insurance_coverage_percent;

resp = invokeUrl
[
    url : "https://<your-catalyst-function-url>/net-fee?consultationFee=" + fee + "&coveragePercent=" + coverage
    type : GET
    connection : "catalystconn"
];

if(resp.containsKey("netFee"))
{
    // Replace net_payable_fee with the actual field link name.
    input.net_payable_fee = resp.get("netFee");
}
```

## 5. Implementation Screenshots

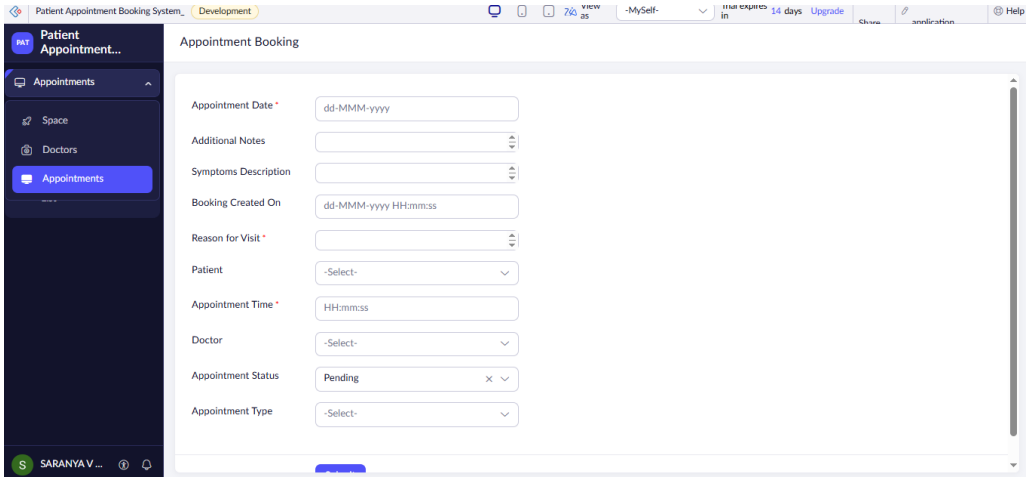



*Figure 1: Appointment Booking Form in the Patient Appointment Scheduling System, showing appointment-specific fields.*

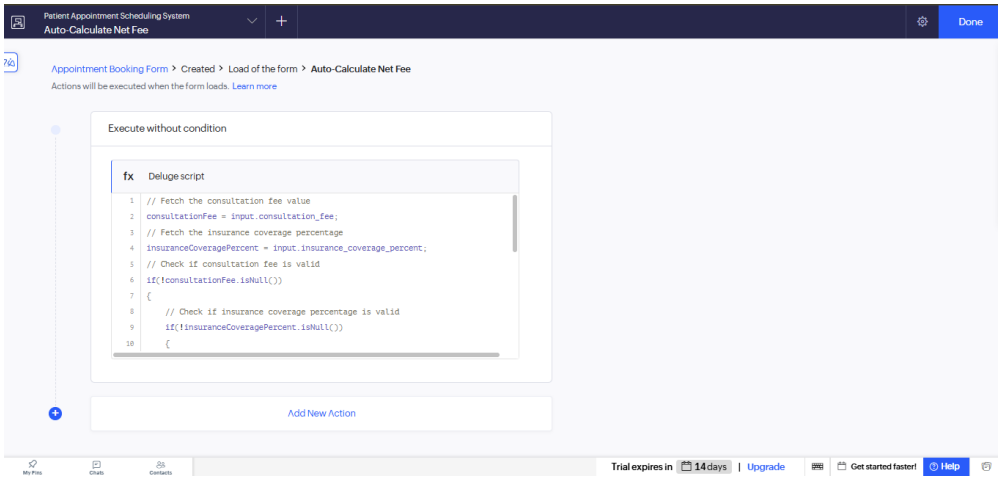



*Figure 2: Existing Auto-Calculate Net Fee workflow and its Deluge script in Zoho Creator.*

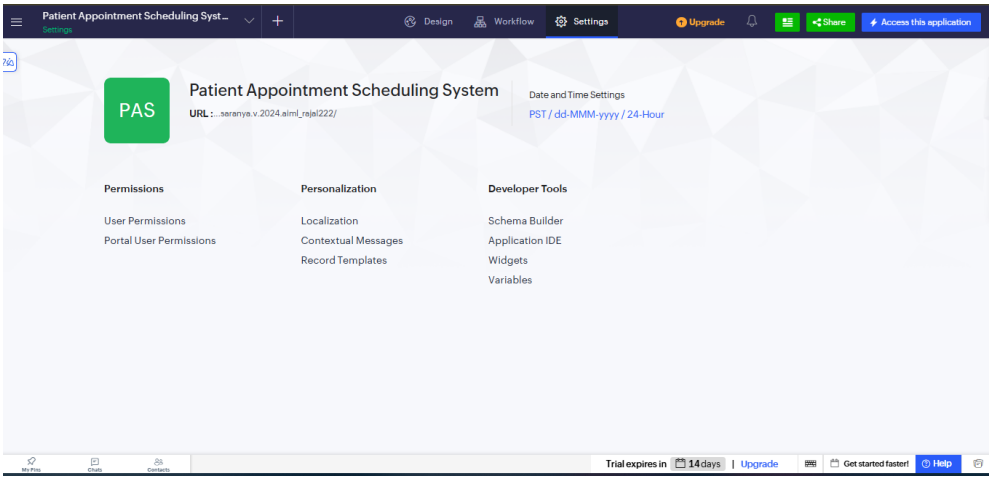



*Figure 3: Application Settings page showing Developer Tools, including Schema Builder, Application IDE, Widgets and Variables.*

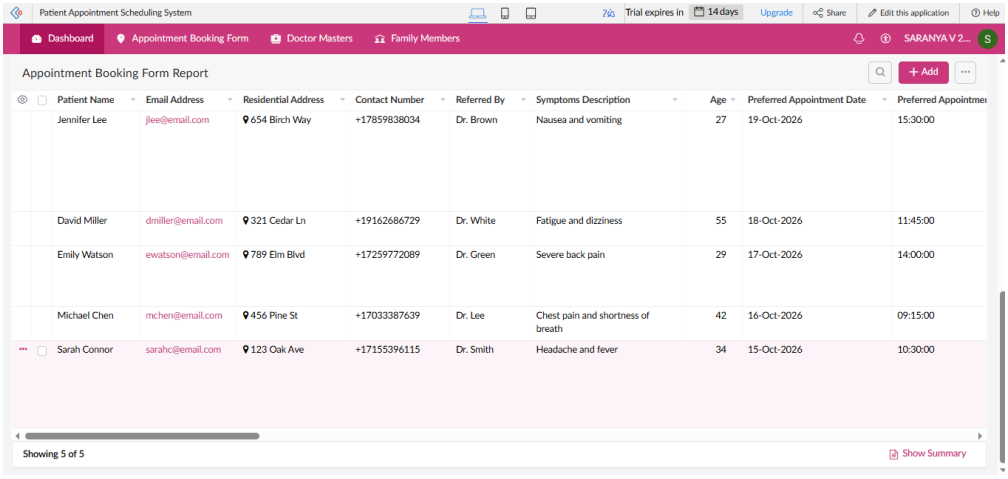



*Figure 4: Appointment Booking Form Report showing saved sample appointment records.*

## 6. Expected Output

For a consultation fee of 1000 and insurance coverage of 20%, the proposed function returns JSON containing netFee: 800. Creator can use the returned value to populate the net payable fee field after the endpoint, connection and field link names are configured correctly.

## 7. Result

The pro-code extension design was adapted to the Patient Appointment Scheduling System. The supplied screenshots confirm the existing form, Deluge workflow, developer tools and report. The Catalyst function and invokeUrl call are proposed code examples; a successful deployment or live integration is not claimed because no Catalyst endpoint or connection test evidence was supplied.

## 8. Learning Outcome

Learnt how existing Deluge workflows can be complemented by reusable functions and an external serverless service, how Creator can call an HTTPS endpoint through invokeUrl, and how to distinguish implemented low-code features from a proposed pro-code extension.